

Lecture 5: GD, Logistic Regression, Classification

Announcements

- HW2 will be released on Canvas by Friday
- Quiz, Report & Code due Mon Oct 5 at 11:59PM ET on Gradescope
 - Programming part Q1, part (d): observe by experiments
 - Q1 Part (e) bonus: need to provide a mathematical proof
 - Q2: clinical trial success rate prediction
 - Part (i) bonus: top 10 on leaderboard, see detailed instruction on the PDF. Key: combine the datasets.

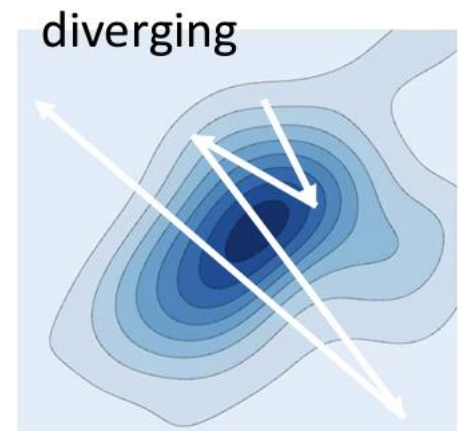
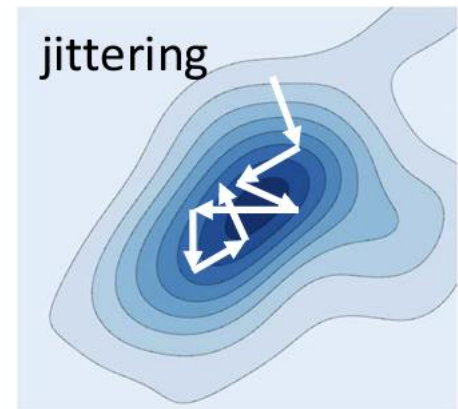
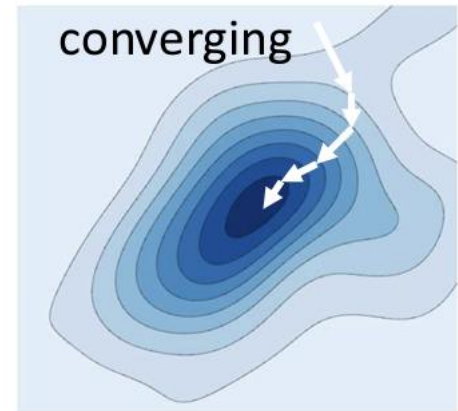
Convergence of Gradient Descent

An iterative algorithm is said to **converge** to a solution if the value updates arrive at a fixed point

- Where the gradient is 0 and further updates do not change the estimate

The algorithm may not actually converge

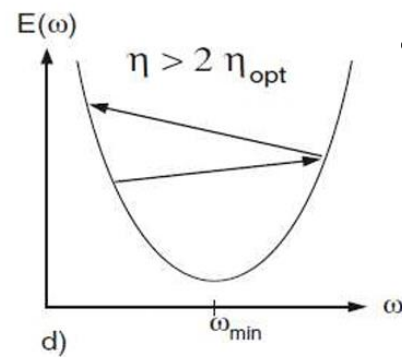
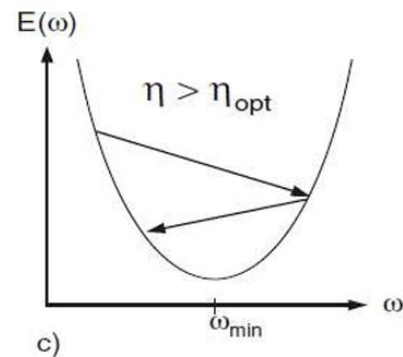
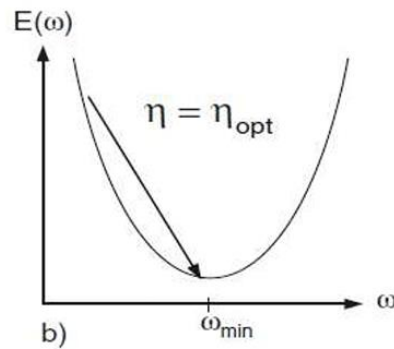
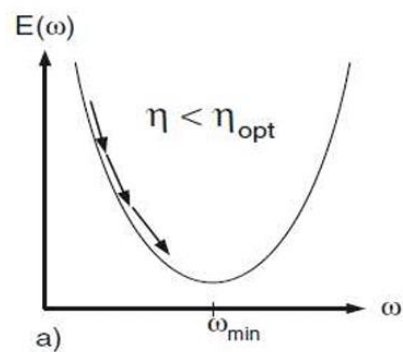
- It may jitter around the local minimum
- It may even diverge
- Conditions for convergence?



With Non-Optimal Step Size

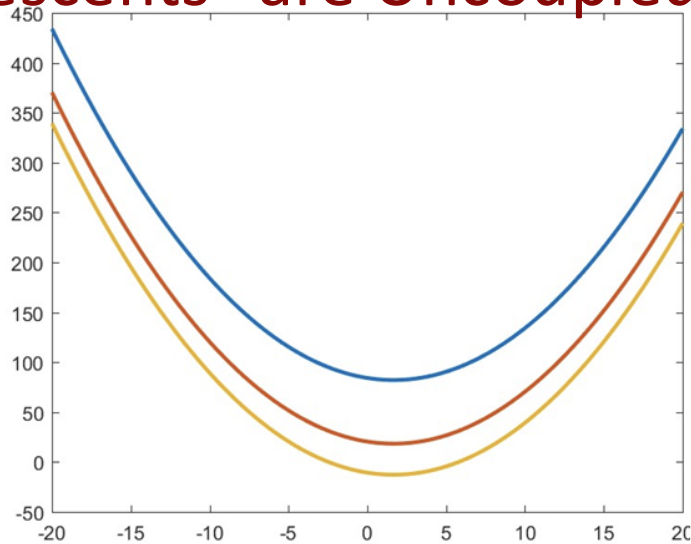
$$w^{(k+1)} = w^{(k)} - \eta \frac{dE(w^{(k)})}{dw}$$

Gradient descent with fixed step size η to estimate *scalar* parameter w

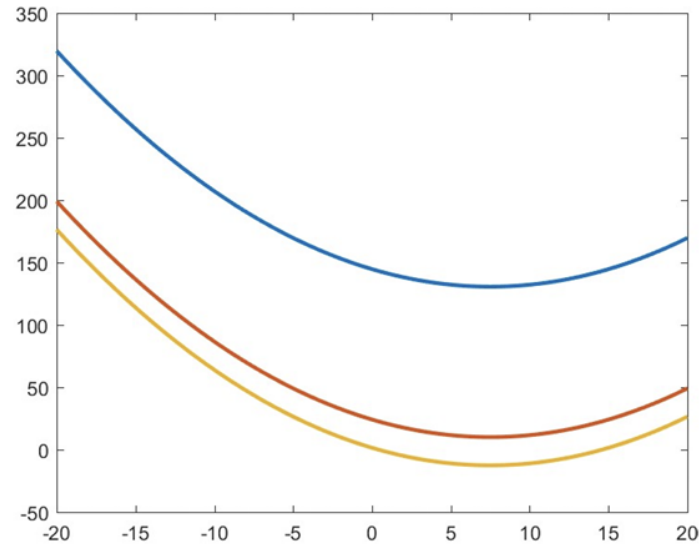


- For $\eta < \eta_{\text{opt}}$ the algorithm will converge monotonically
- For $\eta_{\text{opt}} < \eta < 2\eta_{\text{opt}}$ we have oscillating convergence
- For $2\eta_{\text{opt}} < \eta$ we get divergence

“Descents” are Uncoupled



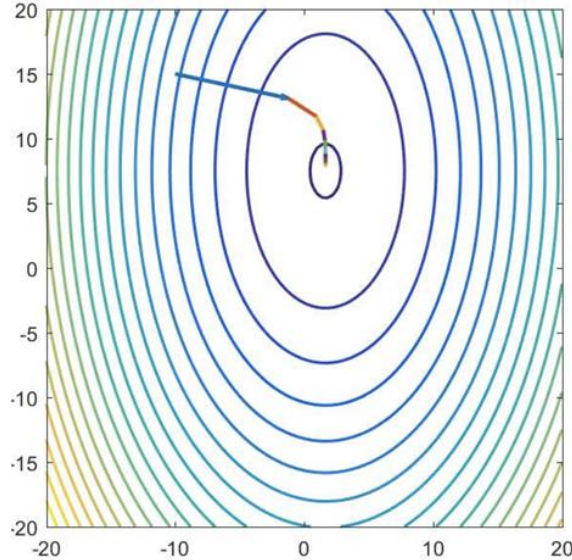
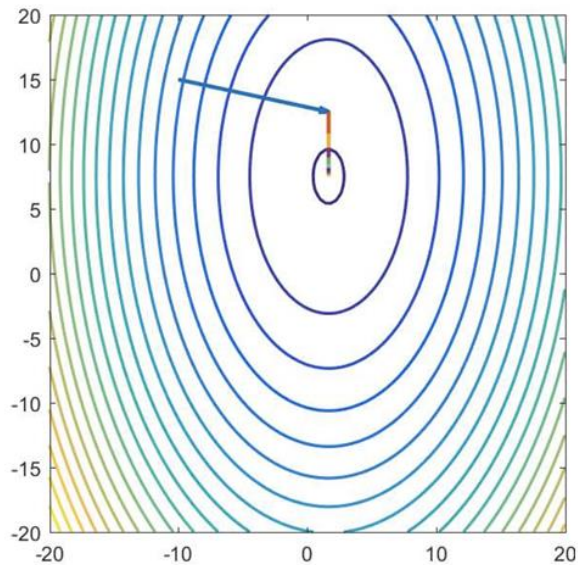
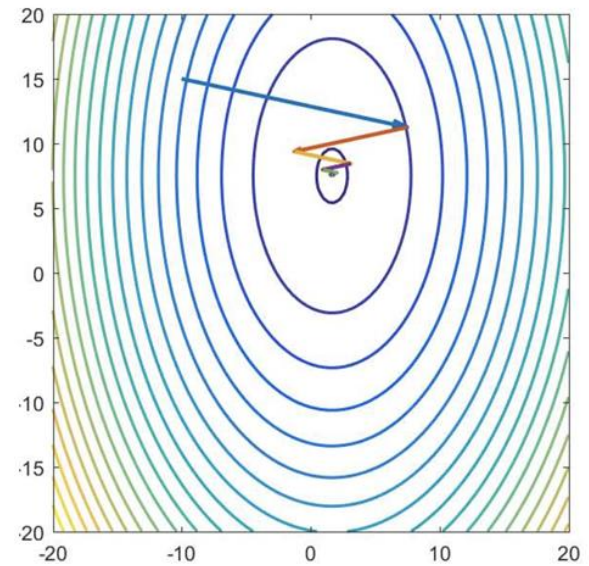
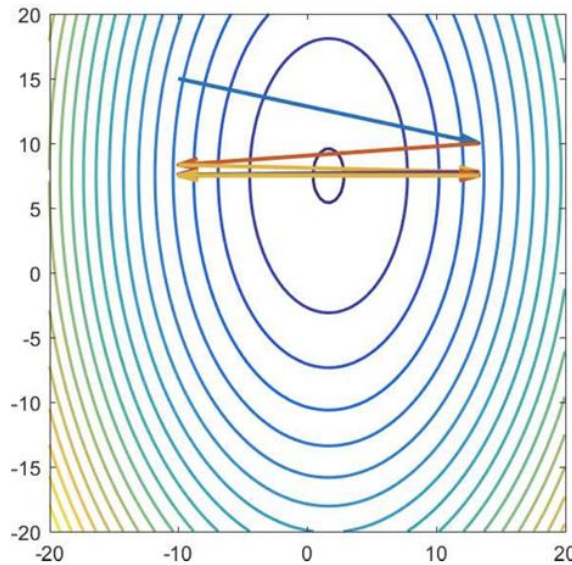
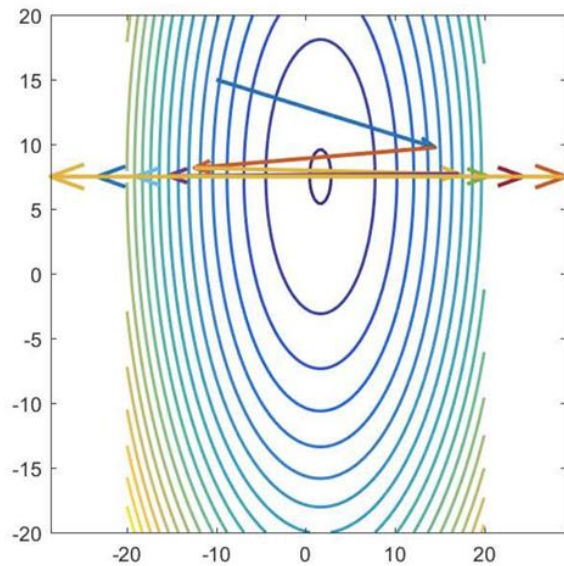
$$E = \frac{1}{2} a_{11} w_1^2 + b_1 w_1 + c + C(\neg w_1)$$
$$\eta_{1,opt} = a_{11}^{-1}$$



$$E = \frac{1}{2} a_{22} w_2^2 + b_2 w_2 + c + C(\neg w_2)$$
$$\eta_{2,opt} = a_{22}^{-1}$$

- The optimum of each coordinate is not affected by the other coordinates
- I.e. we could optimize each coordinate independently
- **Optimal learning rate is different for the different coordinates**

Dependence on Learning Rate



- $\eta_{1,opt} = 1; \eta_{2,opt} = 0.33$
- $\eta = 2.1\eta_{2,opt}$
- $\eta = 2\eta_{2,opt}$
- $\eta = 1.5\eta_{2,opt}$
- $\eta = \eta_{2,opt}$
- $\eta = 0.75\eta_{2,opt}$

Problem with Vector Update Rule

$$\mathbf{w}^{(k+1)} \leftarrow \mathbf{w}^{(k)} - \eta \nabla_{\mathbf{w}} E^T$$

$$w_i^{(k+1)} = w_i^{(k)} - \eta \frac{\partial E(w_i^{(k)})}{\partial w}$$

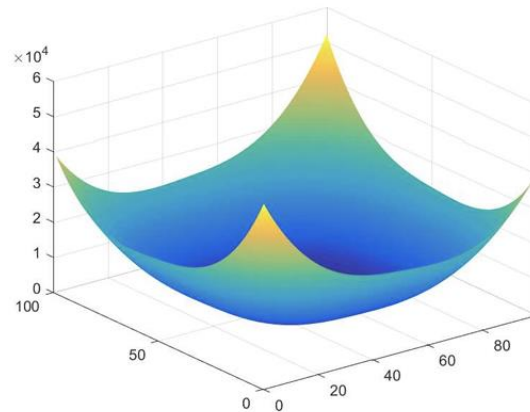
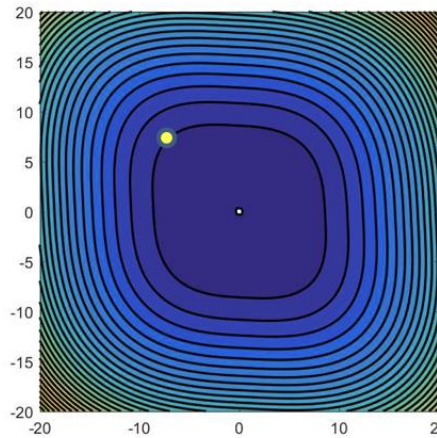
$$\eta_{i,opt} = \left(\frac{\partial^2 E(w_i^{(k)})}{\partial w_i^2} \right)^{-1} = a_{ii}^{-1}$$

- The learning rate must be lower than twice the smallest optimal learning rate for any component

$$\eta < 2 \min_i \eta_{i,opt}$$

- Otherwise the learning will diverge
- This, however, makes the learning very slow
- And will oscillate in all directions where $\eta_{i,opt} \leq \eta < 2\eta_{i,opt}$

Generic Differentiable Multivariate Convex Functions

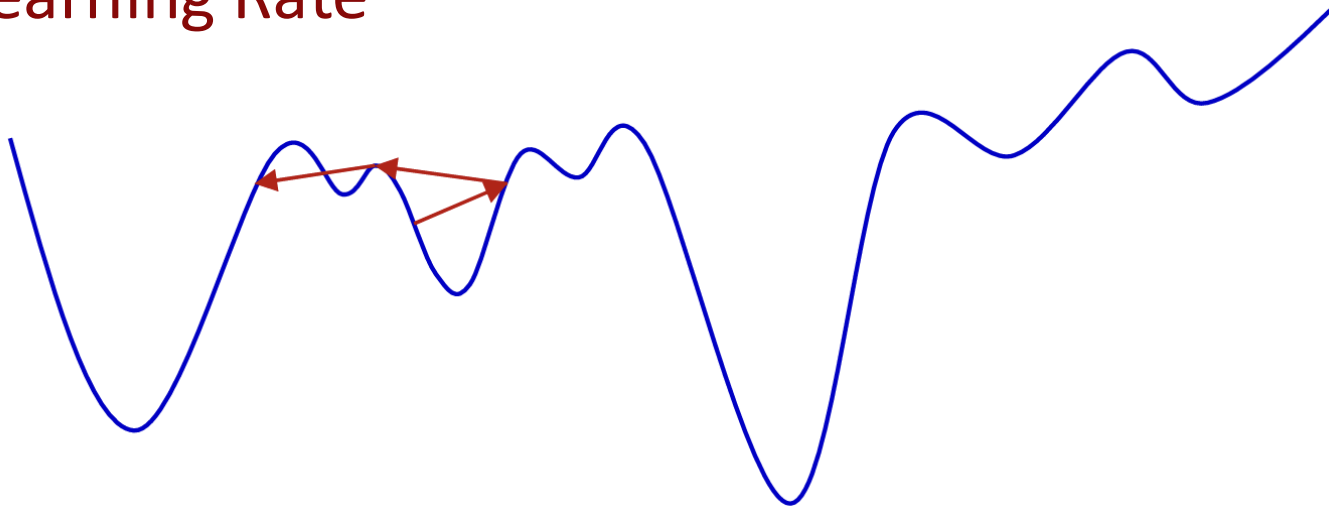


- For generic convex multivariate functions (not necessarily quadratic), we can employ quadratic Taylor series expansions and much of the analysis still applies
- Taylor expansion
$$E(\mathbf{w}) \approx E(\mathbf{w}^{(k)}) + \nabla_{\mathbf{w}} E(\mathbf{w}^{(k)}) (\mathbf{w} - \mathbf{w}^{(k)}) + \frac{1}{2} (\mathbf{w} - \mathbf{w}^{(k)})^T H_E(\mathbf{w}^{(k)}) (\mathbf{w} - \mathbf{w}^{(k)})$$
- The optimal step size is inversely proportional to the Eigenvalues of the Hessian

Convergence

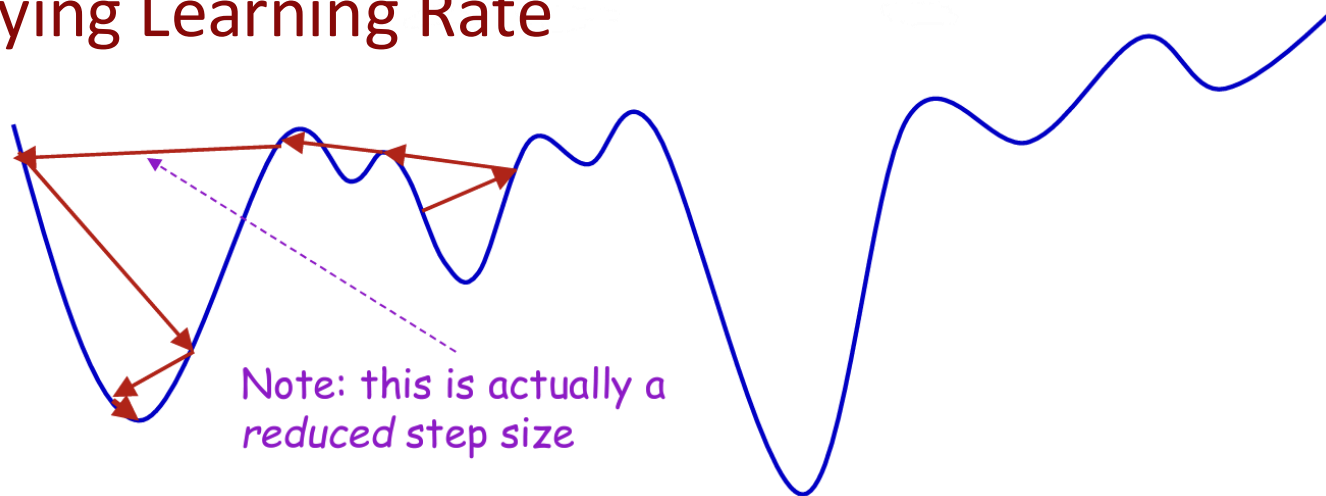
- Convergence behaviors become increasingly unpredictable as dimensions increase
- For the fastest convergence, ideally, the learning rate η must be close to both, the largest $\eta_{i,opt}$ and the smallest $\eta_{i,opt}$
 - To ensure convergence in every direction
 - Generally infeasible
- Convergence is particularly slow if $\frac{\max_i \eta_{i,opt}}{\min_i \eta_{i,opt}}$ is large

The Learning Rate



- For complex models the loss function is often not convex
- Having $\eta > 2\eta_{opt}$ can actually help escape local optima
- However always having $\eta > 2\eta_{opt}$ will ensure that you never ever actually find a solution

Decaying Learning Rate



- Start with a large learning rate
- Greater than 2
- Gradually reduce it with iterations
- Typical decay schedules

- Linear decay: $\eta_k = \frac{\eta_0}{k+1}$

- Quadratic decay: $\eta_k = \frac{\eta_0}{(k+1)^2}$

- Exponential decay: $\eta_k = \eta_0 e^{-\beta k}$, where $\beta > 0$

Select all correct statements

0

Step sizes that are greater than twice the inverse of the second derivative can cause gradient descent to diverge

0

Diverging gradient is always a bad thing

0

Gradient descent will not converge without decaying learning rates

0

When the dataset size is large, we always want to start with a relatively small learning rate so that gradient descent will not explode

0

When the dimension of the dataset is large, we always want to start with a relatively small learning rate so that gradient descent will not explode

0

When the dimension of the model is large, we always want to start with a relatively small learning rate so that gradient descent will not explode

0

Story So Far: Convergence

Gradient descent can miss obvious answers

- And this may be a good thing

Convergence issues abound

- The loss surface has many saddle points
 - Gradient descent can stagnate on saddle points
- Vanilla gradient descent may not converge, or may converge tooooooo slowly
- The optimal learning rate for one component may be too high or too low for others

Solution

Advanced optimizer schemes

- Stochastic gradient descent (SGD)
 - Approximate the gradient at each step using either 1 sample (true SGD) or a few (minibatch SGD)
 - Noisy but unbiased
 - Still guaranteed to converge to the global minimum for convex loss surfaces
 - Much faster to compute the gradient at each step since I am only looking at a subset of the data each time

Solution

Advanced optimizer schemes

- Other optimizers
 - <https://pytorch.org/docs/stable/optim.html>
 - torch.optim contains a list of algorithms that you can try out by yourself

- | | |
|--------------|-----------|
| ▪ Adadelta | ▪ NAdam |
| ▪ Adagrad | ▪ RAdam |
| ▪ Adam | ▪ RMSprop |
| ▪ AdamW | ▪ Rprop |
| ▪ SparseAdam | ▪ SGD |
| ▪ Adamax | |
| ▪ ASGD | |
| ▪ LBFGS | |

```
for input, target in dataset:
    optimizer.zero_grad()
    output = model(input)
    loss = loss_fn(output, target)
    loss.backward()
    optimizer.step()
```

Take Away

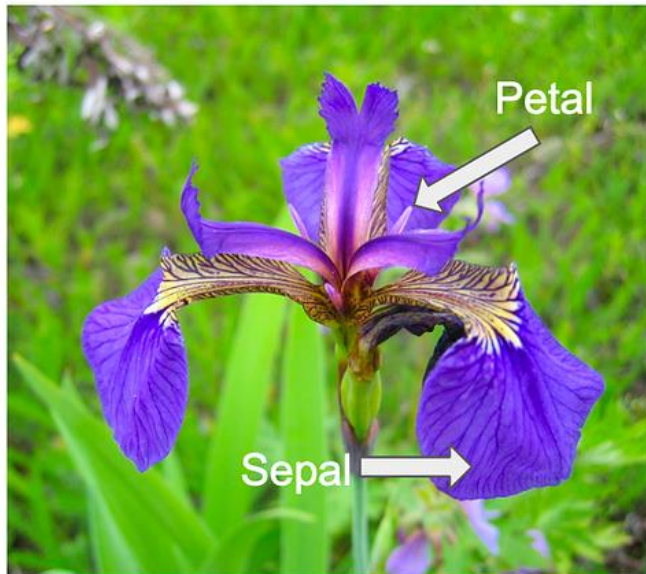
- Given a fixed dataset, the loss function is only a function of the model parameter θ
- We can use gradient descent as an optimization technique to identify the “best” θ that minimizes the loss function

Logistic Regression

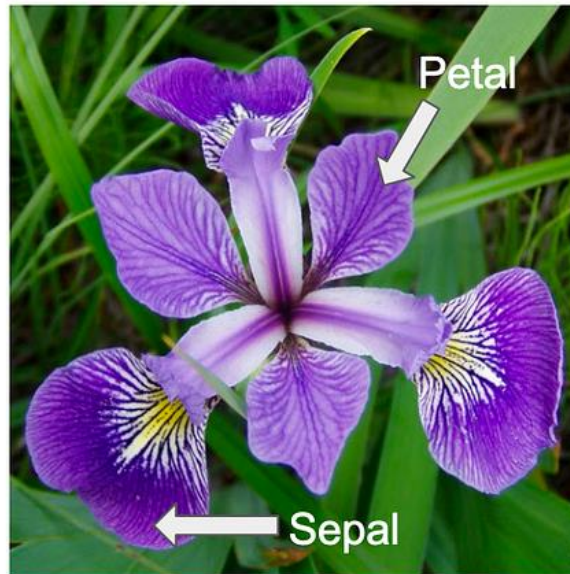
Binary Classification Refresher

- Binary classification: target variable value $y \in \{0,1\}$

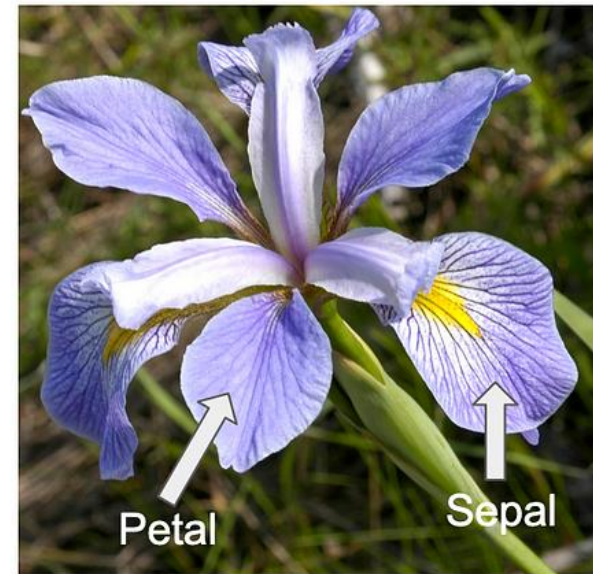
Iris setosa



Iris versicolor



Iris virginica



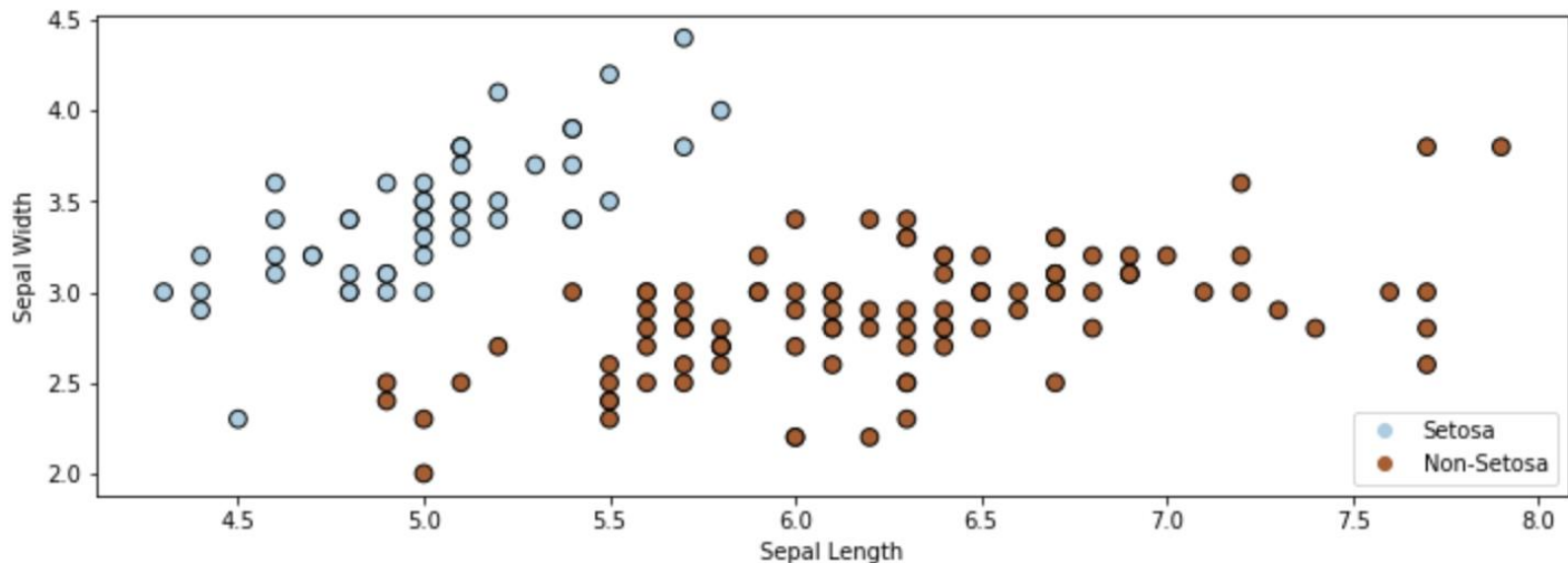
- Distinguishing class 0 (Iris Setosa) from the other two classes

Binary Classification Refresher

- Binary classification: target variable value $y \in \{0,1\}$

Example

- Iris flower dataset



- Distinguishing class 0 (Iris Setosa) from the other two classes

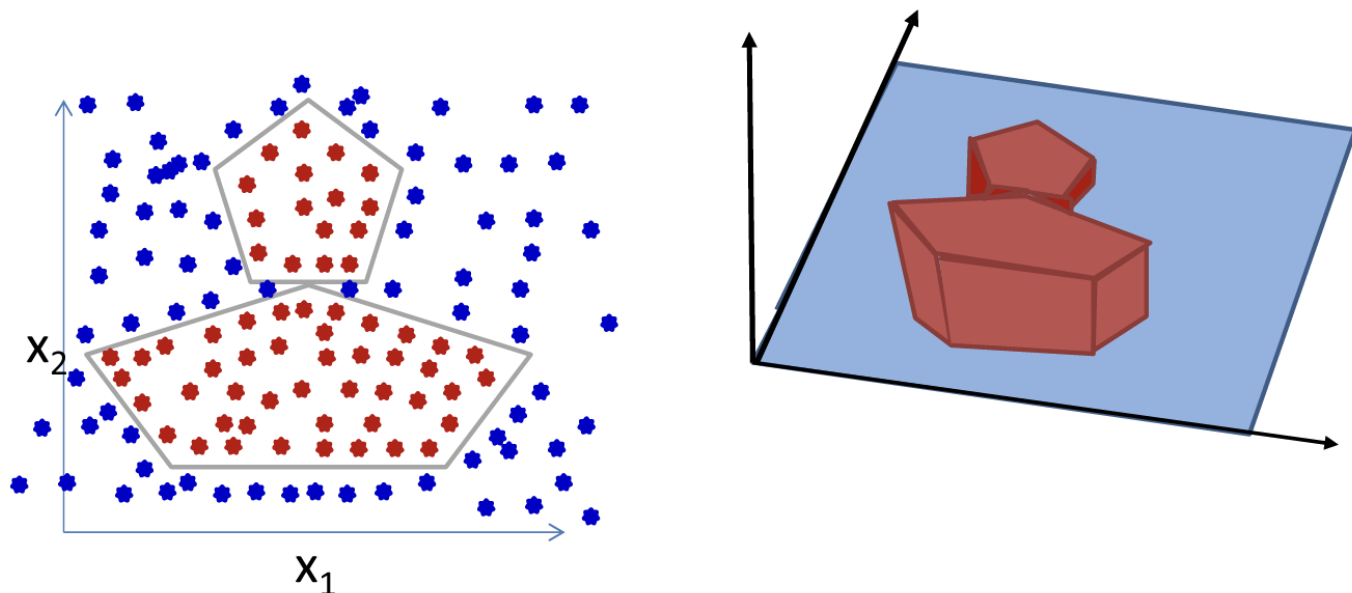
Classification VS Regression

In regression

- Goal: fit a curve through the output space that passes close to the set of targets $y^{(i)}$.

In classification

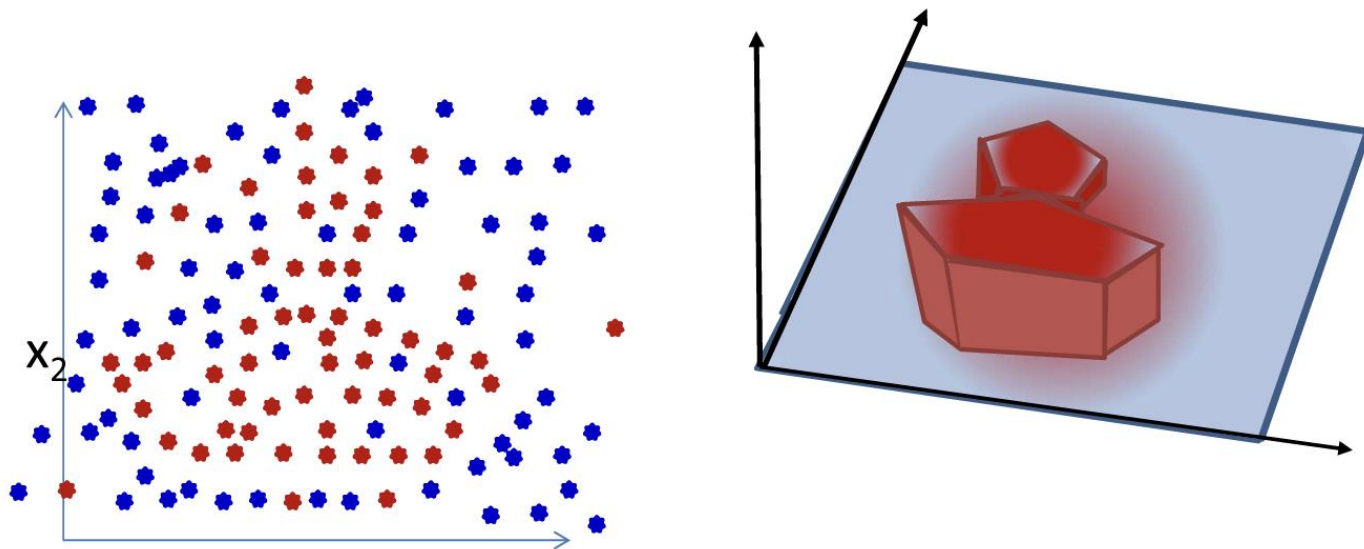
- Classes are associated with regions of the feature space



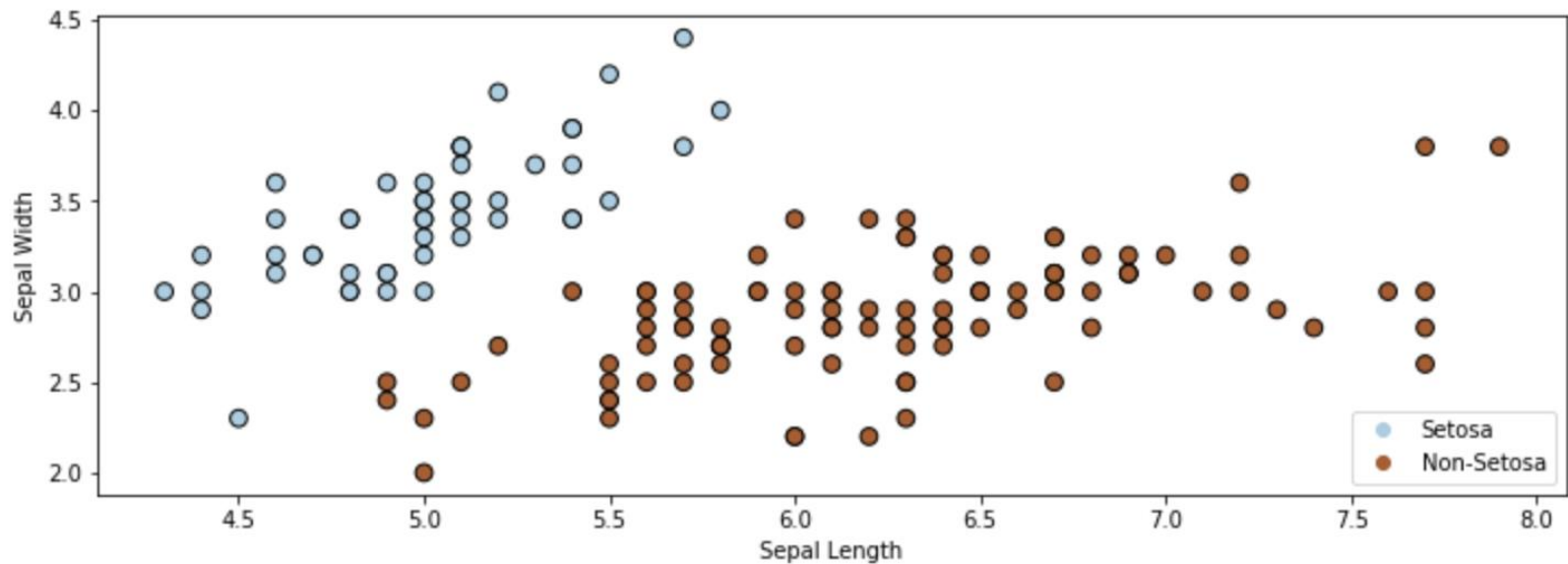
Classification VS Regression

In classification, classes are associated with regions of the feature space

- Goal: find the boundaries between these regions
- Output of classification models:
 - Represent the probabilities that a data point belongs to a given class



First Attempt: Linear Regression

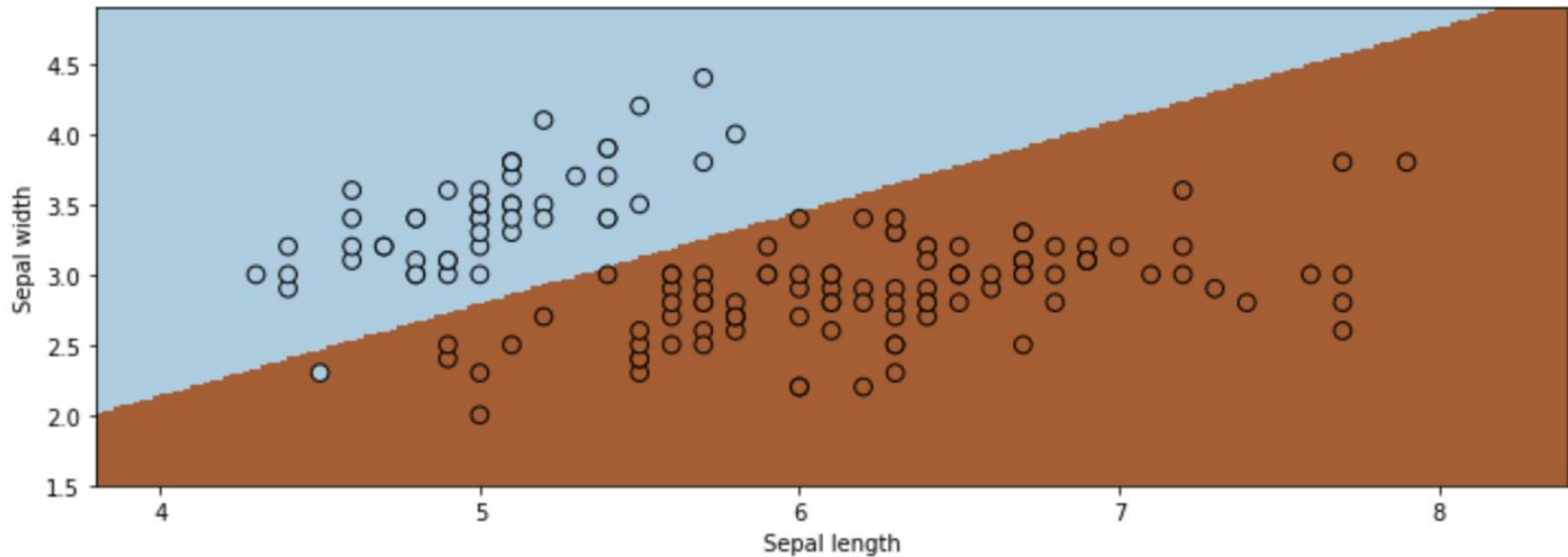


- Setting $\mathcal{Y} = \{0,1\}$.
- Run a linear regression to fit the data:

$$Y = \theta^T X$$

And minimizes it using MSE

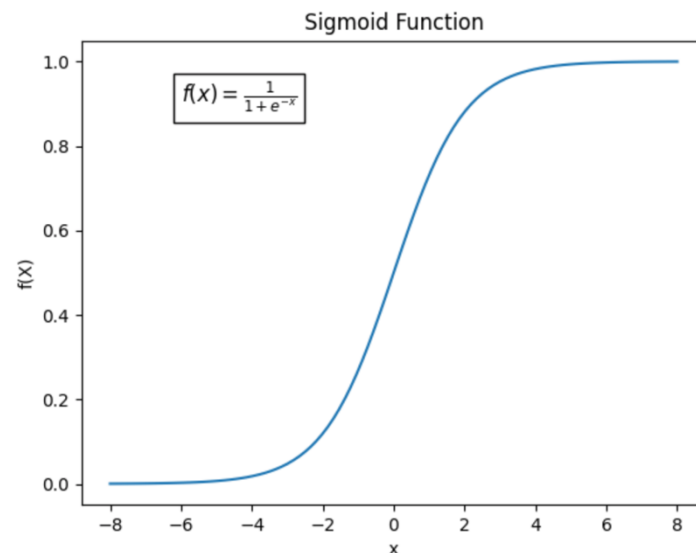
First Attempt: Linear Regression



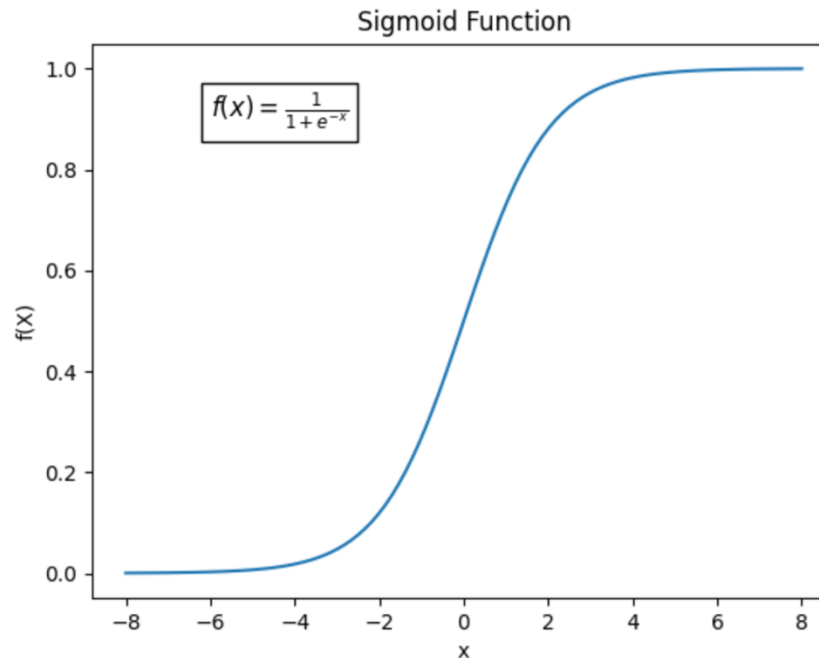
- Setting $\mathcal{Y} = \{0,1\}$.
- Color plots:
 - Predict 1 if regression outcome > 0.5
 - 0 if regression outcome < 0.5

Issues with Linear Regression

- The predicted output $\hat{y}_i$ might be negative
- There is no guarantee that the output will be bounded between 0 and 1
- Solution: pass the output through a sigmoid function, to force the output to be between 0 and 1



Logistic Regression: The Model

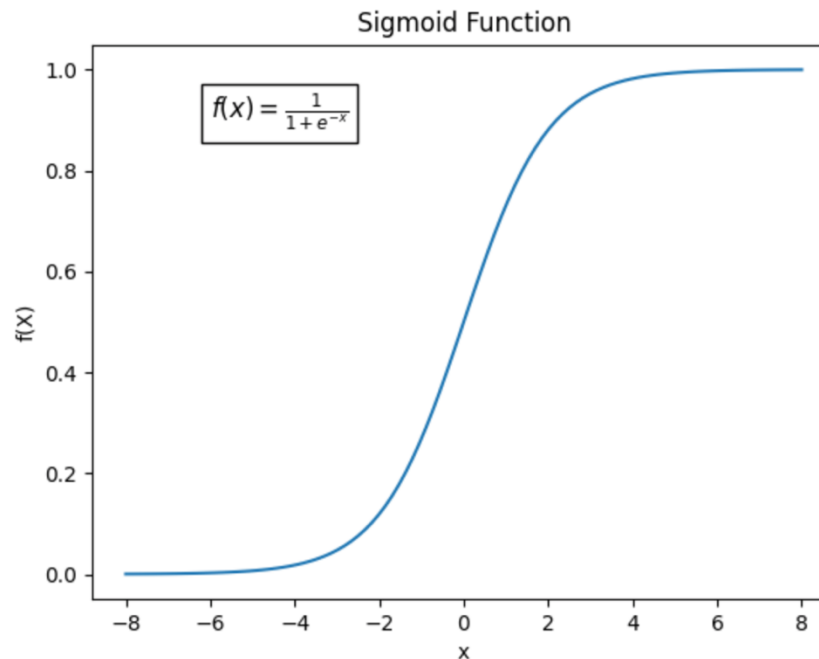


Sigmoid function: $\sigma(z) = \frac{1}{1+\exp(-z)}$

Logistic regression: $f_{\theta}(x) = \sigma(\theta^T x) = \frac{1}{1+\exp(-\theta^T x)}$

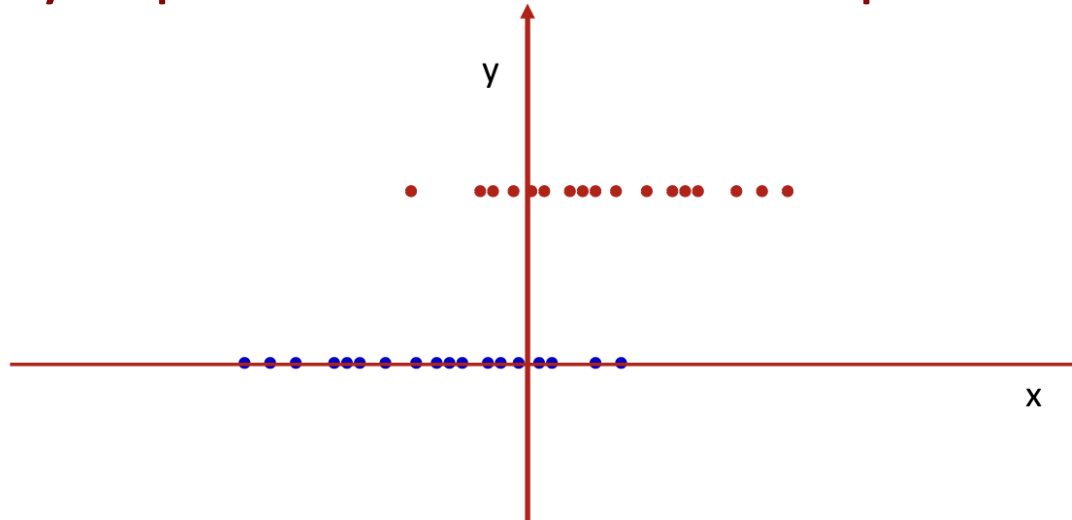
This is a binary classification algorithm

The Logistic Function: Properties



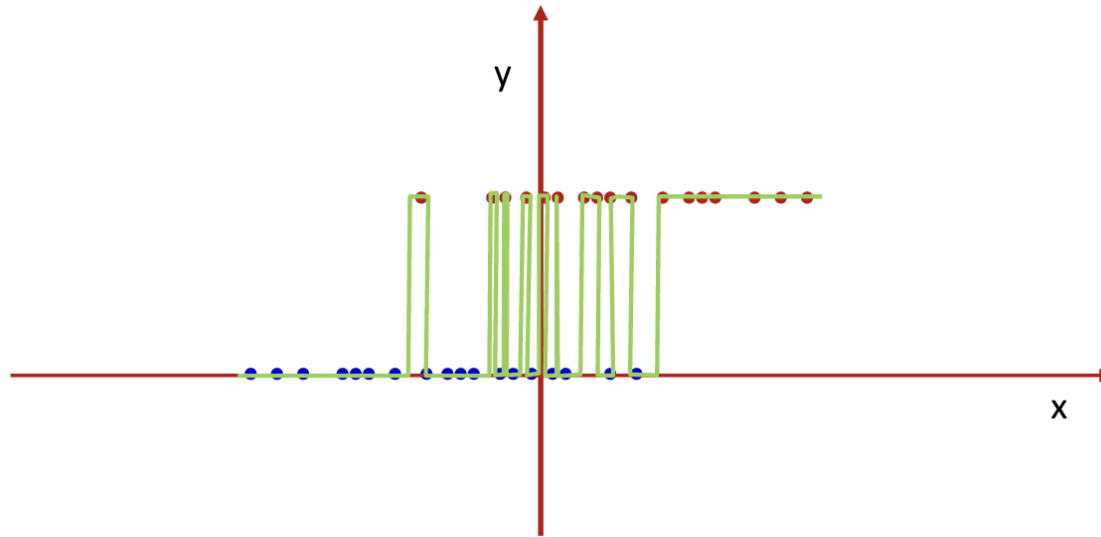
- The function is bounded between 0 and 1
 - Tends to 1 as $z \rightarrow \infty$, and 0 as $z \rightarrow -\infty$
- The output resembles probability

Non-linearly separable data: 1-D example



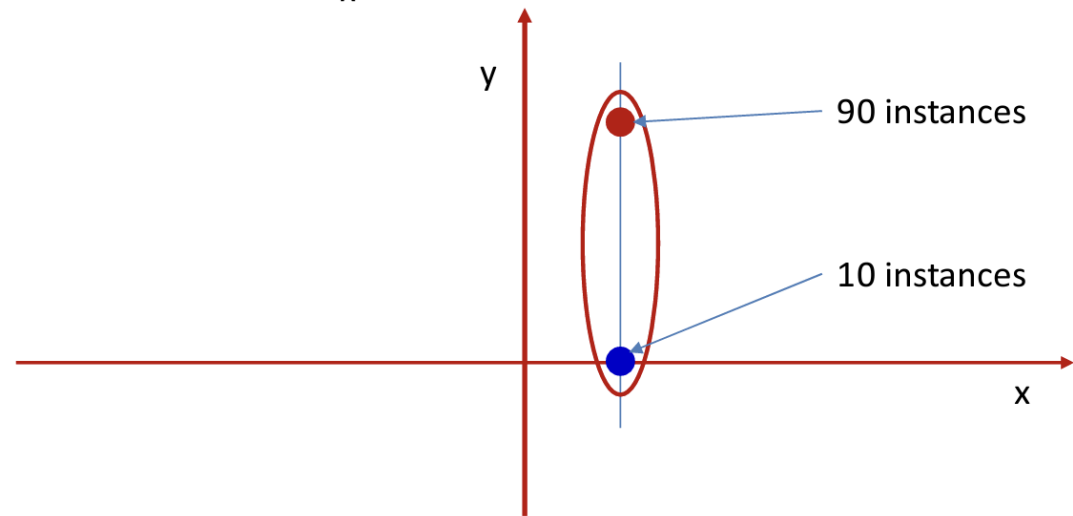
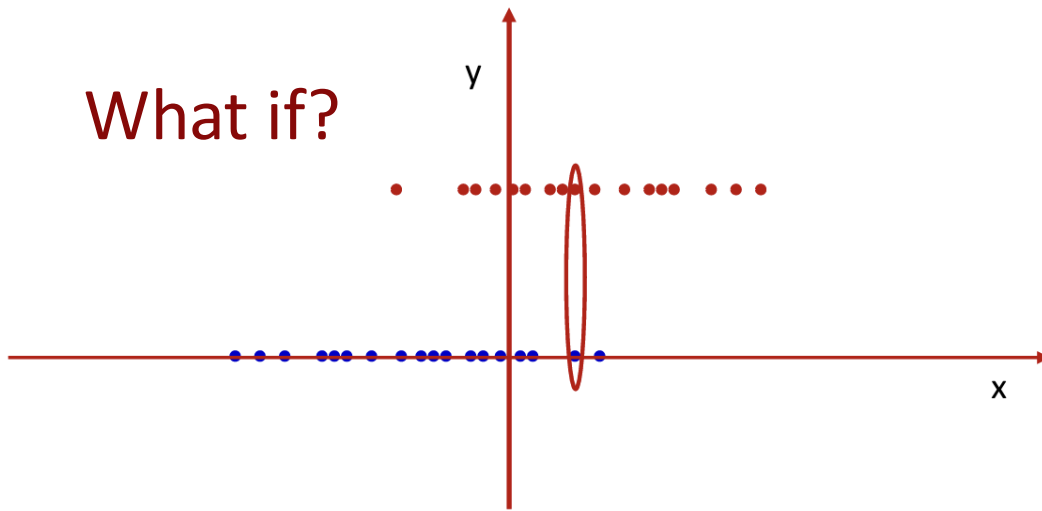
- One-dimensional example for visualization
 - All (red) dots at $Y = 1$ represent instance of class $Y = 1$
 - All (blue) dots at $Y = 0$ are from class $Y = 0$
- The data are not linearly separable
 - In this 1-D example, a linear separator is a threshold
 - No threshold will cleanly separate red and blue dots

Undesired Function



- One-dimensional example for visualization
 - All (red) dots at $Y = 1$ represent instance of class $Y = 1$
 - All (blue) dots at $Y = 0$ are from class $Y = 0$
 - The data are not linearly separable
 - In this 1-D example, a linear separator is a threshold
 - No threshold will cleanly separate red and blue dots

What if?



- What must the value of the function be at this X?
- 1 because red dominates?
- 0.9: the average?

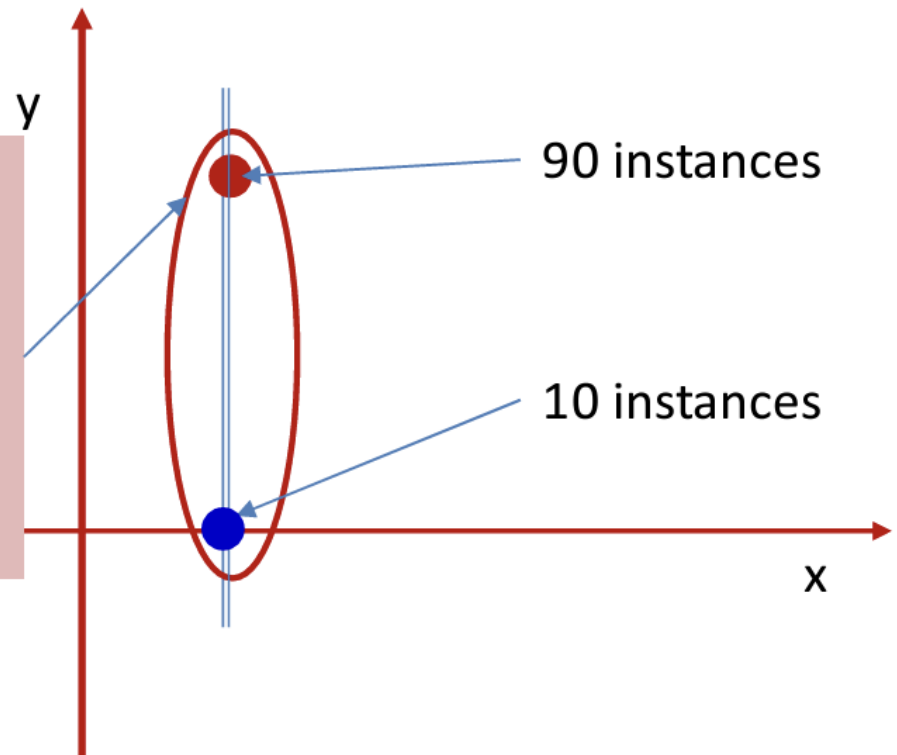
Estimate: $\approx P(Y = 1|X)$

Potentially much more useful than
a simple 1/0 decision
Also, potentially more realistic

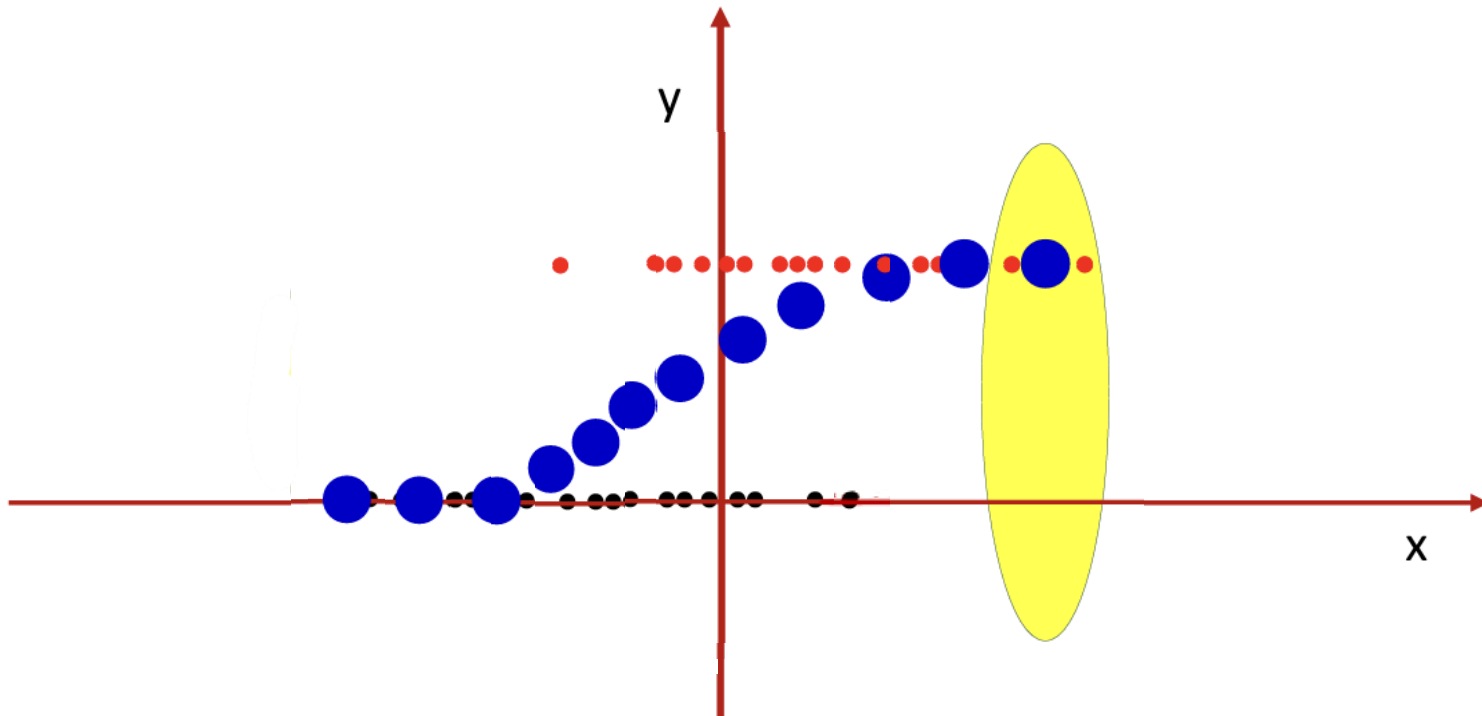
What if?

Should an infinitesimal nudge of the red dot change the function estimate entirely?

If not, how do we estimate $P(1|X)$?
(since the positions of the red and blue X Values are different)



The Probability of $Y = 1$

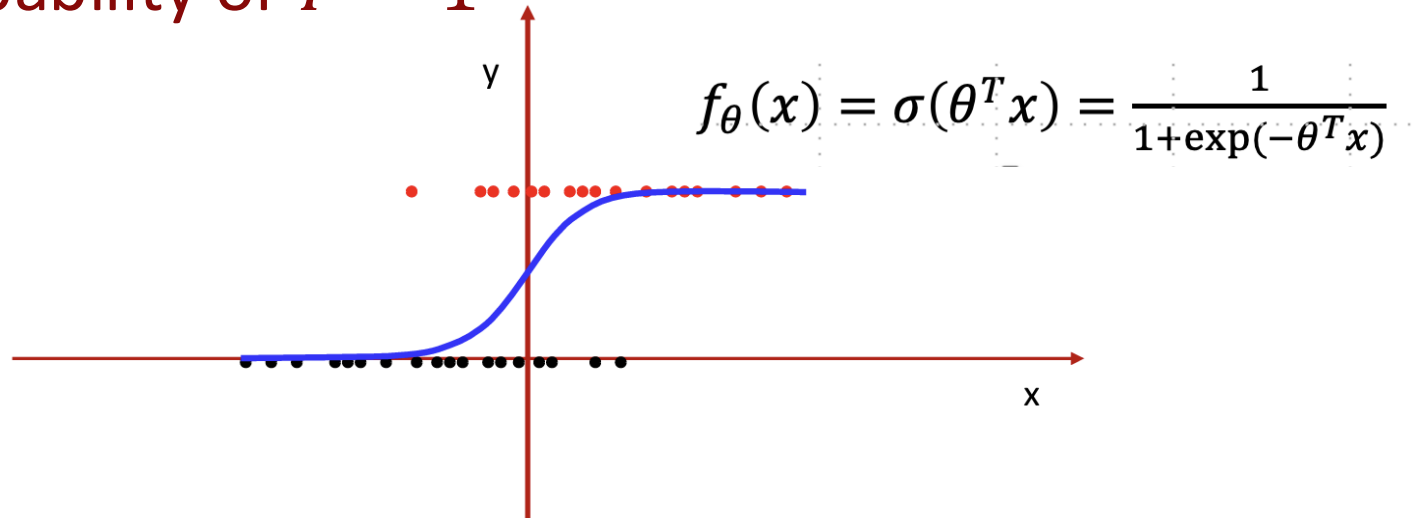


Consider this differently: at each point look at a small window around that point

Plot the average value within the window

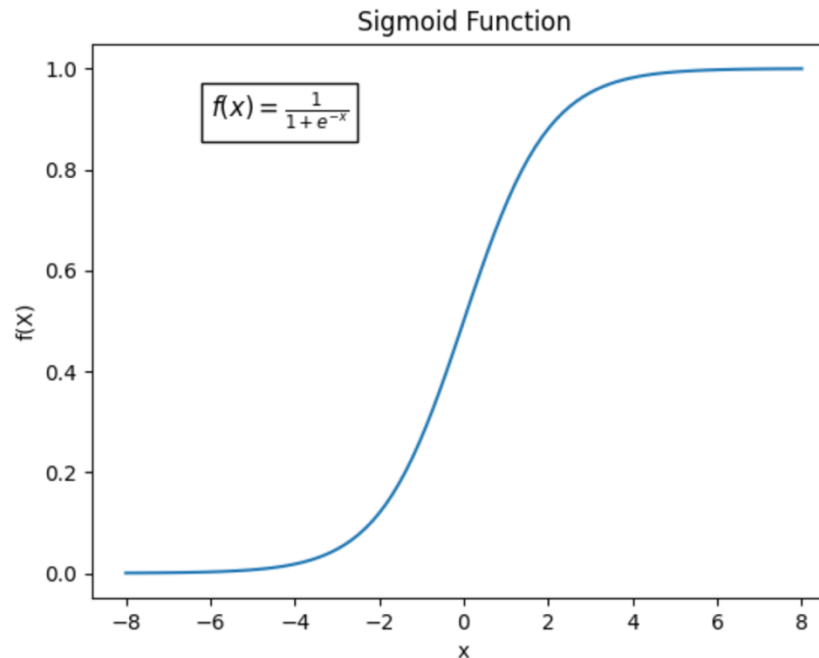
- This is an approximation of the probability of $Y = 1$ at that point

The Probability of $Y = 1$



- Class 1 becomes increasingly probable going left to right
- Given the training data $\mathcal{D} = \{(X_i, y_i) | i = 1, 2, \dots, n\}$, estimate θ_0 and θ_1 for the curve
- Given θ_0 and θ_1 , can calculate how probable the actual classes are given the features
 - What is the probability of seeing $Y = 1$ given input X here?
 - What about $Y = 0$?

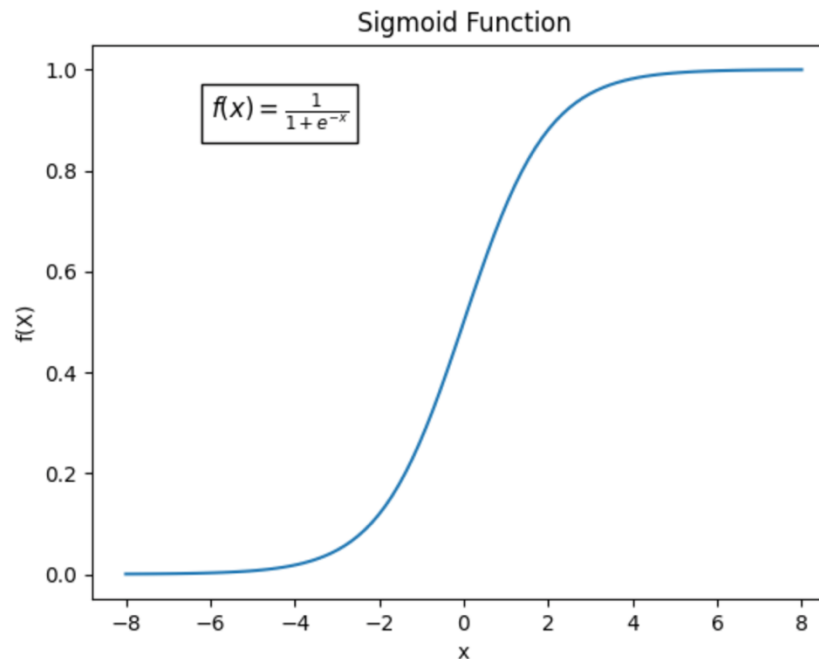
The Logistic Function: Probabilistic Interpretations



- Output can be interpreted as a probability

$$P_{\theta}(y = 1|X) = \sigma(\theta^T X)$$
$$P_{\theta}(y = 0|X) = 1 - \sigma(\theta^T X)$$

The Logistic Function: Properties



- Sigmoid is easy to take derivative $\frac{d\sigma}{dz} = \sigma(z)(1 - \sigma(z))$
- Can use gradient descent to optimize

How Do We Optimize Logistic Regression?

- Which optimizer?
 - Some variants of gradient descent
- Which objective function?
- What is my prediction $\hat{y}_i$?
 - I can threshold my output probability to make the output binary
 - The error $|\hat{y}_i - y_i|$ is not continuous
 - Small perturbation in the model parameter might not yield any change in the error!

How Do We Optimize Classification?

- Need to think about the likelihood of seeing this data given the model parameters instead
- Within my model class, want to find the set of parameters that are most likely to **generate** the data
- **Maximum Likelihood Estimation** defines the “best fit”

Story so far

- Given a loss function differentiable with respect to model parameters, we can always optimize it using gradient descent
- To define the loss function, we need the concept of maximum likelihood estimator
- To find the set of parameters that are **most likely to generate** the data
- Maximum likelihood estimator defined the loss function in many cases!

Select all correct statements

Logistic regression learns a linear decision boundary between 2 classes

0

Logistic regression can also learn nonlinear decision boundaries (nonlinear with respect to features)

0

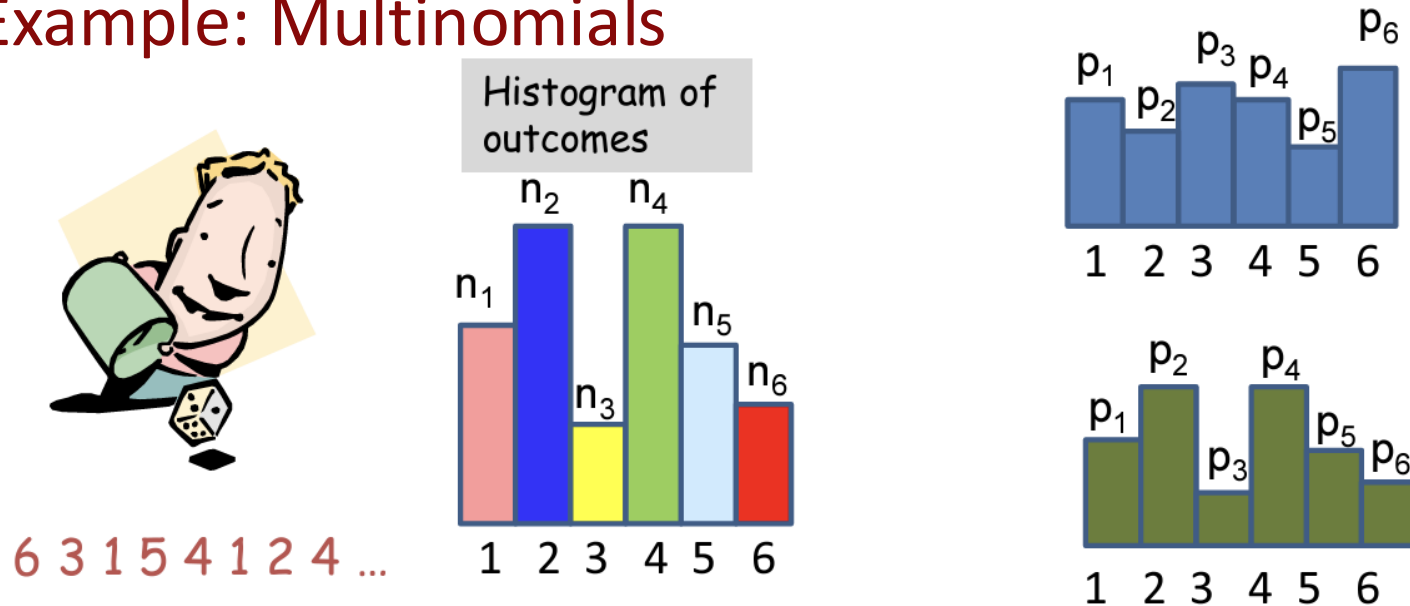
We can derive closed-form optimal parameter for logistic regression given data, just like in a linear regression

0

The Maximum Likelihood Principle

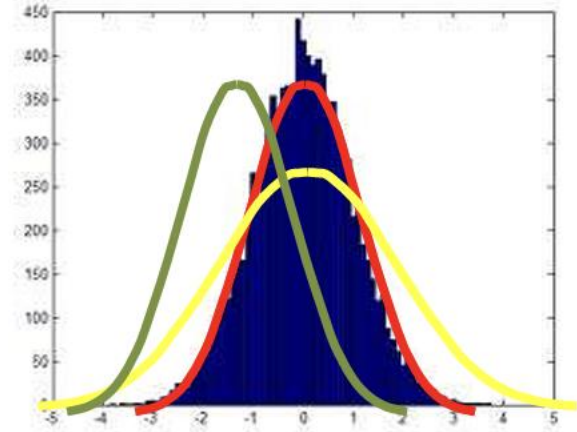
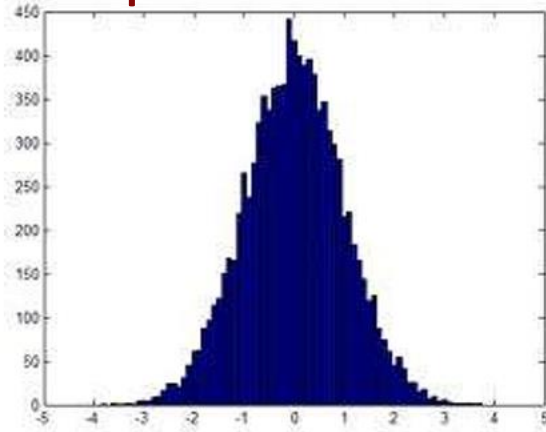
- You are given some set of observed data $\mathcal{D} = \{(X_i, y_i) | i = 1, 2, \dots, n\}$
- You choose a model $P_\theta(y|X)$ for the distribution of $y|X$
 - θ are the parameters of the model
- Estimate the θ such that $P_\theta(y|X)$ best “fits” the observations $\mathcal{D}$
 - Hoping it will also represent data outside the training set.

An Example: Multinomials



- A dice roller rolls dice and you plot the histogram of outcomes (Shown to the right)
- The distribution is a multinomial
- Parameters to be learned: $p_1, p_2, p_3, p_4, p_5, p_6$
- Which of the two probability distributions shown to the right is more likely to be the distribution for the dice?
- Why?

An Example



- The left figure shows the histogram of a collection of observations
- We decide to model the distribution as Gaussian
 - Parameters: Mean μ and variance σ^2
- Which of the three Gaussians shown in the right figure is most likely to be the actual PDF of the RV?
- Why?

Defining the “Best fit”: Maximum Likelihood

- The data are generated by draws from the distribution
 - I.e., the generating process draws from the distribution
- Assumption: The world is a boring place
 - The data you have observed are very typical of the process
- Consequent assumption: The distribution has a high probability of generating the observed data
 - Not necessarily true
- Select the distribution that has the highest probability of generating the data
 - Should assign lower probability to less frequent observations and vice versa

The Maximum Likelihood Principle

- You are given some set of observed data $\mathcal{D} = \{(X_i, y_i) | i = 1, 2, \dots, n\}$
- You choose a model $P_\theta(y|X)$ for the distribution of $y|X$
 - θ are the parameters of the model
- Estimate the θ such that $P_\theta(y|X)$ best “fits” the observations $\mathcal{D}$
 - Hoping it will also represent data outside the training set.

Estimating the Model: Logistic Regression

- Given the training data $\mathcal{D} = \{(X_i, y_i) | i = 1, 2, \dots, n\}$
- Total probability of data (assuming independence)
- $P((X_1, y_1), \dots, (X_n, y_n))$

$$= \prod_{i=1}^n P(X_i, y_i)$$

Express this term in terms of the things that we know

$$= \prod_{i=1}^n P(y_i | X_i) P(X_i)$$

$$= \prod_{i=1}^n [y_i \sigma(\theta^T X_i) + (1 - y_i)(1 - \sigma(\theta^T X_i))] P(X_i)$$

Estimating the Model: Logistic Regression

- Given the training data

$$\begin{aligned} P((X_1, y_1), \dots, (X_n, y_n)) \\ &= \prod_{i=1}^n [y_i \sigma(\theta^T X_i) + (1 - y_i)(1 - \sigma(\theta^T X_i))] P(X_i) \\ &= \prod_{i=1}^n [y_i \sigma(\theta^T X_i) + (1 - y_i)(1 - \sigma(\theta^T X_i))] \prod_{i=1}^n P(X_i) \end{aligned}$$

How to make this easier to calculate?

- Linearize this by taking the logarithm

Estimating the Model

$$\log(ab) = \log(a) + \log(b)$$

- Given the training data

$$P((X_1, y_1), \dots, (X_n, y_n))$$

$$= \prod_{i=1}^n [y_i \sigma(\theta^T X_i) + (1 - y_i)(1 - \sigma(\theta^T X_i))] P(X_i)$$

$$= \log \left(\prod_{i=1}^n [y_i \sigma(\theta^T X_i) + (1 - y_i)(1 - \sigma(\theta^T X_i))] \prod_{i=1}^n P(X_i) \right)$$

$$= \log \left(\prod_{i=1}^n [y_i \sigma(\theta^T X_i) + (1 - y_i)(1 - \sigma(\theta^T X_i))] \right) + \log \left(\prod_{i=1}^n P(X_i) \right)$$

$$= \sum_{i=1}^n \log(y_i \sigma(\theta^T X_i) + (1 - y_i)(1 - \sigma(\theta^T X_i))) + \sum_{i=1}^n \log P(X_i)$$

Estimating the Model

- Given the training data

$$\begin{aligned} & \log \left(P((X_1, y_1), \dots, (X_n, y_n)) \right) \\ &= \log \left(\prod_{i=1}^n [y_i \sigma(\theta^T X_i) + (1 - y_i)(1 - \sigma(\theta^T X_i))] P(X_i) \right) \\ &= \log \left(\prod_{i=1}^n [y_i \sigma(\theta^T X_i) + (1 - y_i)(1 - \sigma(\theta^T X_i))] \prod_{i=1}^n P(X_i) \right) \\ &= \sum_i \log [y_i \sigma(\theta^T X_i) + (1 - y_i)(1 - \sigma(\theta^T X_i))] \\ & \quad + \sum_i \log P(X_i) \end{aligned}$$

fixing the data. This is a constant!

Maximum Likelihood Estimation

- Log Likelihood

$$P((X_1, y_1), \dots, (X_n, y_n)) = \prod_{i=1}^n P(y_i | X_i) P(X_i)$$

$$\log \left(P((X_1, y_1), \dots, (X_n, y_n)) \right)$$

$$= \sum_i \log[y_i \sigma(\theta^T X_i) + (1 - y_i)(1 - \sigma(\theta^T X_i))] + \sum_i \log P(X_i)$$

- Maximum likelihood estimation

$$\hat{\theta} = \arg \max_{\theta} \log P((y_1, X_1), \dots, (y_n, X_n))$$

- Focusing on the bits that invoke the parameters

$$\hat{\theta} = \arg \max_{\theta} \log P((y_1 | X_1), \dots, (y_n | X_n))$$

$$\hat{\theta} = \arg \max_{\theta} \sum_i \log[y_i \sigma(\theta^T X_i) + (1 - y_i)(1 - \sigma(\theta^T X_i))]$$

$$\hat{\theta} = \arg \min_{\theta} \sum_i -\log[y_i \sigma(\theta^T X_i) + (1 - y_i)(1 - \sigma(\theta^T X_i))]$$

Select all correct statements

0

Maximum-likelihood estimation of probability distributions is based on the theory that the world is a terribly boring place

0

Maximum-likelihood estimation estimates the values of the parameters of a probability distribution such that they maximize the probability of the training data

0

Maximum Likelihood Estimate

- Equals (note argmin rather than argmax)

$$\hat{\theta} = \arg \min_{\theta} \sum_i -\log[y_i \sigma(\theta^T X_i) + (1 - y_i)(1 - \sigma(\theta^T X_i))]$$

- Identical to minimizing the **KL divergence** between the desired output y and actual output $\sigma(\theta^T X)$
- Maximum likelihood learning of a logistic minimizes the KL divergence between its output and the target output
- Cannot be solved directly, needs gradient descent